

Research Brief 02 — Free / Low-Cost LLM Infrastructure for Agentic Coding

A model supply strategy for a Claude Code-like agent harness at zero or near-zero cost

Author: Manus AI • Verification date: August 8–9, 2026 (all quotas, pricing, and provider terms verified against official documentation at time of writing; pricing in this ecosystem rots in weeks, so treat every number as a snapshot, not a promise)

1. Executive Summary

The central question of this investigation — *"Can we construct a capable coding-agent model stack using free and inexpensive models instead of paying for a single premium model?"* — has a clear, evidence-based answer: **yes, with real limitations, and the architecture matters more than any individual model**. The findings that led there are summarized below and expanded throughout this report.

The most important structural fact discovered during verification is that **the free API ecosystem is actively dying and being rebuilt at the same time**. In the three months before this research, GitHub Models was permanently retired (July 30, 2026) ⁹, OpenRouter delisted more than half of its free models within nine days ⁸, Alibaba discontinued the Qwen Code free tier (April 2026) ¹⁰, and Cerebras silently shrank its free catalog from roughly a dozen models to two without any deprecation notice, breaking a real production pipeline with a 404 ¹¹. Any strategy that hard-wires a harness to a single free endpoint will be broken by a provider decision within months. The correct strategy is therefore a **router-fronted, multi-provider, capability-pinned supply chain**, and nearly every recommendation in this report follows from that fact.

On the model side, the news is genuinely good. Open-weight coding models have reached the point where a small Mixture-of-Experts model with only **3 billion active parameters (Qwen3-Coder-Next, 80B total) scores over 70% on SWE-bench Verified with a real agent scaffold** — territory that only frontier commercial models occupied two years ago ³. In independent small-task agentic evaluations, the free-tier models **Gemma 4 26B-a4b and Qwen 3.5 35B-a3b match Claude Sonnet's score (9/10)** ¹². The strongest free-tier models are no longer toys; they are yesterday's flagships.

The bottleneck is token economics, not raw intelligence. A 50-turn agentic session sends roughly **25:1 input-to-output token ratio, with input tokens driving 85–90% of the cost**, and a full Claude Code-class session costs roughly ten times more on a frontier model than on a cheap one (\$6.00 vs \$0.60 for one session) ¹³. Every dollar of budget therefore buys far more engineering capability when it flows through a router that sends trivial work to

\$0.01/M flash models and reserves expensive models for genuinely hard turns. Benchmarks from two commercial routing vendors report **30–65% cost reductions at equal or better outcomes**, and a coding-agent-native router reportedly solved 71/89 Terminal-Bench tasks for \$65.75 where a single frontier model solved 63/89 for \$190.62 ¹⁴.

The user's hardware (i5-8250U, 8GB RAM, integrated graphics) is the binding constraint for local inference: it can run ~4–8B models at 3–8 tokens/sec, which is **too slow for a primary coding loop but excellent for offline fallback, summarization, classification, and context compression**. The practical role of the laptop is a support tier, not a compute tier — a conclusion the evidence supports with near certainty.

The recommended core architecture is: a **LiteLLM proxy self-hosted on the laptop** in front of four tiers — (1) permanent free primary tiers (Gemini 3.5 Flash free, Cerebras free, Mistral free), (2) free heavy-lift tiers (Groq, Cloudflare Workers AI, NVIDIA NIM, OpenRouter :free), (3) ultra-cheap paid (\$0.01–0.30/M: Qwen 3.x flash, DeepSeek V4 Flash, Mistral), and (4) local models as summarizer/classifier/fallback. The harness speaks only to the LiteLLM proxy at localhost:4000 ; no provider can strand the agent when it changes terms.

Verdict line	Answer
Best free model	gemini-3.5-flash (free tier) — quality closest to paid frontier among free, 1M TPM, ~1,500 RPD, permanent tier
Best free provider	Groq (deepest stable free catalog, no card, fast) — but dual-home everything
Best coding model	Qwen3-Coder family / GLM-5.2 for free-lane; frontier: Claude Sonnet 5 / GPT-5.x
Best reasoning model	Gemini 3.5 Flash free tier; paid: Claude Opus-class / Qwen 3.7 Max
Best cheap model	Mistral small (\$0.02–0.03/M) or Qwen 3.x flash (\$0.05–0.10/M)
Best local model	qwen2.5-coder:7b Q4 (coding), phi-4-mini 3.8B (classification) — support roles only
Best router	LiteLLM proxy (self-hosted, free, quota-aware, Anthropic-compatible) + OpenRouter as discovery surface
Best overall strategy	Harness ↔ LiteLLM ↔ four tiers, failover-first, cache-aware, capability-pinned

The decisive strategic conclusion (Section 16) is that **effort is better spent on the harness than on hunting the best model**: at every realistic budget level from \$0 to \$10/month, orchestration quality, context management, and routing discipline produce larger capability differences than swapping models within a tier. The model only defeats the harness when it is too weak to sustain a 50+ turn tool loop at all — the failure boundary identified in Section 16.3.

2. The Current Model Landscape

2.1 Frontier models (the performance ceiling)

Frontier models establish the ceiling that the free/cheap stack should be measured against. As of mid-2026, the frontier landscape is dominated by Anthropic and OpenAI, with Google and xAI close behind. On Vellum's aggregated coding benchmarks (March 2026 data), **Claude Sonnet 4.5 leads SWE-bench Verified at 82%**, followed by Claude Opus 4.5 (80.9%), Claude Opus 4.6 (80.8%), GPT-5.2 (80%), and Claude Sonnet 4.6 (79.6%) ¹. By August 2026 the newest generation has moved these numbers further — independent leaderboards now show Claude Opus 5 at 96–97% and GPT-5.6 at ~96% on SWE-bench Verified ¹⁵, though the interpretation of those numbers requires caution (Section 5).

Model family	Representative score	Strength profile	Free access path
Claude 5 series (Anthropic)	Sonnet 4.5: 82% SWE-bench V (Mar) ¹ ; Opus 5: ~96% (Aug) ¹⁵	Best agentic tool loops, strongest engineering judgment	~\$5 trial only; startup program up to \$25K ¹⁶
GPT-5.x series (OpenAI)	GPT-5.2: 80% (Mar) ¹ ; GPT-5.6: ~96% (Aug) ¹⁵	Strong reasoning, strong Terminal-Bench scores	~\$5 trial; startup credits via VC referral only ¹⁶
Gemini 3.x (Google)	Gemini 3 Pro: 79.7% LiveCodeBench ¹	Very large context, strong all-round	Permanent free tier on 3.5 Flash + lite models ²
Grok 3.x (xAI)	79.4% LiveCodeBench (3 Beta) ¹	Fast, strong open coding	\$25 signup credit + data-sharing program ¹⁷
Kimi K3 (Moonshot)	"Best open-weight coding model" claims (Jul 2026) ¹⁸	Largest open-weight model (2.8T params)	API access only

The key strategic point about frontier models is what they do *not* offer a budget-constrained user: **Anthropic and OpenAI have no permanent free API tier** — only one-time trials of roughly \$5 — and their meaningful credit programs (Anthropic Startup up to \$25K, OpenAI ~\$2,500 via VC referral) target funded startups, not individuals ¹⁶. The frontier matters here as a ceiling reference and as an occasional burst spend, not as a sustained supply.

2.2 Strong open-weight models

The open-weight ecosystem underwent a step change in late 2025 and 2026, driven by two architectural trends: **Mixture-of-Experts models with tiny active parameter counts** (which fit modest hardware and cost little to serve) and **post-training specifically on long-horizon agent trajectories** (which matters more than raw size for agent tasks).

Model	Architecture	Context	Cost	Evidence
Qwen 3.5 397B-A17B	397B/17B active	~1M	API	GPQA Diamond 88.4%, AIME 2026 91.3% ¹
Qwen 3.7 Max	Dense, API	1M	API, \$/M-class	SWE-bench Pro 60.6% ¹
DeepSeek-V4-Pro	1.6T/49B active, MIT	1M	\$0.27/\$0.40 (rising) ¹⁹	80.6% SWE-bench Verified claimed ¹
DeepSeek-V4-Flash	284B/13B active, MIT	1M	\$0.14/\$0.28 ¹⁹	Fast, cheap production workhorse
GLM-5.2 (Z.ai)	753B/40B active	1M	~\$0.95/\$3.00 direct ²⁰	Terminal-Bench 2.1: 81.0%; built for multi-hour engineering tasks ¹
Kimi K2.5–K2.6 / K3	1T/32B active MoE; K3: 2.8T	1M+	API	K3 largest open-weight ever (Jul 2026) ¹⁸
Qwen3.6 27B	Dense	256K (1M)	Cheap/free-hosted	SWE-bench Verified 77.2%, Terminal-Bench 2.0: 59.3% ¹

Gemma 4 26B-a4b	26B/~4B active, Apache 2.0	256K	Free-hosted	LiveCodeBench v6 77.1%, AIME 2026 88.3% 1
Llama 4 Scout/Maverick	109B/17B; 400B/17B	10M (Scout, iRoPE)	Free-hosted	Maverick LiveCodeBench 43.4% 1

Two observations from this table are strategically decisive. First, **the Apache 2.0 / Modified-MIT open models (Qwen 3.6 27B, Gemma 4 26B-a4b, Qwen3-Coder 30B) are exactly the models that free providers choose to host**, which is why the free lane today effectively offers yesterday's \$100/M models at \$0/M. Second, practical community testing confirms that 24–30B MoE models deliver 80–90% of flagship performance from months prior, and the rule for local deployment is to pick the model the hardware can run at usable speed, not the biggest model that nominally fits 5 .

2.3 Coding-specialized models

Coding-specialized models are a distinct and growing category, and the evidence strongly indicates **they genuinely outperform general reasoning models of similar size on agentic coding tasks**. The clearest proof is Qwen3-Coder-Next: an 80B-parameter hybrid (attention + MoE) model with only 3B active parameters, trained on 300-turn long-horizon agent trajectories, that reaches **70.6% on SWE-bench Verified and 44.3% on SWE-bench Pro — matching models ten to twenty times its size** 3 4 . Because its active parameter count is 3B, it is architecturally deployable on commodity hardware in principle (though the full 45GB Q4 footprint is not — Section 10).

Other strong coding-specialized entrants include GLM-5.2, trained with RL over long multi-hour engineering sessions and hitting 81.0% on Terminal-Bench 2.1 1 20; Qwen3-Coder-30B (30.5B total / 3.3B active), tuned specifically for agentic coding and the best all-round local coding model in community tests at ~220 tokens/sec 5 ; and Cohere's north-mini-code, a small model hosted **free on OpenRouter** that is specifically built for code generation and tool use 8 .

The practical implication for the stack: prefer a coding-specialized model for the **coder lane** (patch generation, diff writing) and reserve general models for planning and reasoning, because the specialization effect is real at the sizes free tiers actually offer, not just at frontier scale.

3. The Free API Landscape (Verified August 2026)

3.1 The verified free-tier matrix

Every entry below was verified against official provider documentation during this research (dates noted). The single most important structural warning, repeated throughout this report, is that **this table expires**. Treat it as a snapshot of record and re-verify before deploying.

Provider	Free offer	Quotas (verified)	Key models	Card	Permanence	Commercial use
Google Gemini 27	Permanent free tier on select models	~15 RPM, ~1M TPM, ~1,500 RPD (flash), RPD resets midnight Pacific	gemini-3.5-flash, 3.5-flash-lite, 3.1-flash-lite	No	Permanent tier (documented)	Yes, within free quota; content may train Google products
Groq 21	Permanent free tier, deepest catalog	30 RPM; 14,400 RPD (8B) / 1,000 RPD (70B+); 6–12K TPM	llama-3.3-70b, gpt-oss-120b/20b, qwen3.6-27b, llama-4-scout	No	Stable; catalog churns slowly	Yes
Cerebras 22	Free tier, all hosted models	TPM-based (~30K TPM snapshot), faster TOFT than OpenAI/Antropic	gpt-oss-120b, zai-glm-4.7 (catalog volatile — was ~12 models)	No	Permanent tier, catalog unstable	Yes
Mistral 11	Free/dev tier	~50K TPM snapshot (May 2026), per-model caveats	magistral (reasoning), devstral (coding)	No	Most stable catalog observed	Yes with caveats
Cloudflare Workers AI 23	10,000 neurons/day free	Resets daily 00:00 UTC; ≈ ~300–400K input tokens/day on small models;	qwen3-30b-a3b, glm-5.2, kimi-k2.5/2.6/2.7-code, gpt-oss-120b, gemma-4-26b-a4b	No	Permanent daily allowance	Yes (Terms of Service limits apply)

		~20–30 requests/day on 70B-class				
NVIDIA NIM / Build ²⁴	1,000 inference credits (up to 5,000 on request), 90 days	~40 RPM, credit-metered	glm-5.2, laguna series, nemotron 3 Ultra, qwen family	No (phone verification)	Trial, time-limited — not permanent	Yes
OpenRouter: free ⁸	28+ free endpoints	50 RPD base; 1,000 RPD after one-time \$10 top-up; 20 RPM	nemotron-3-ultra, gemma-4-26b, cohere-north-mini-code, laguna, gpt-oss	\$10 once for full access	High churn: 20→15→14 models in 3 weeks	Yes within limits
Hugging Face ²⁵	\$0.10/month compute (free); Inference Router free	Few hundred requests/hour on community API; free router	Thousands of models via router	No	Permanent small allowance	Yes
xAI ¹⁷	\$25 signup credit; data-sharing program	Data-sharing: spend \$5 → ~\$150/month credits (reported)	Grok 3.x family	For data-sharing program	Promotional/recurring with conditions	Data-sharing terms apply
Alibaba DashScope ²⁰	1M free tokens per model (one-time)	Per-model token grant	glm-5.2, qwen3.7 series	By region	One-time grant	Yes
DeepSeek ¹⁹	No current API free tier	Concurrency: 500 (V4-Pro) / 2,500 (V4-Flash)	V4-Pro, V4-Flash	N/A	Paid only, pricing rising	Yes

Cohere 11	Trial key	1,000 calls/month total, 20 RPM	command-a (agent-built, 256K ctx)	No	Trial	Non-commercial only
------------------	-----------	---------------------------------	-----------------------------------	----	-------	----------------------------

Three models deserve specific attention because they thread the needle between "free" and "actually useful for coding agents." **Gemini 3.5 Flash on the free tier** is the strongest genuinely free primary engine: it carries frontier-adjacent quality, a 1M TPM ceiling that no other free tier approaches, and roughly 1,500 requests per day — in practice the only free model that can absorb a real agentic coding workload without constant backpressure 2 7 . **GPT-OSS-120b** deserves special status as the ecosystem's dual-homing champion: it is an open-weight 120B model hosted free on *both* Groq and Cerebras (and NVIDIA), so a lane built around it survives any single provider's catalog deletion without output drift 11 22 . And **Cohere north-mini-code** is a small coding-specialized model free on OpenRouter that is worth using for the coder lane on trivial turns 8 .

3.2 What died: the fragility record

This research documented four concrete failures of free providers in mid-2026, and each one shapes the architecture recommendations later. **GitHub Models was fully retired on July 30, 2026** — a major card-carrying free provider that offered Phi-4, Mistral Large, and Jamba without payment card simply ceased to exist 9 . **OpenRouter's free model count fell from 20 to 14 in three weeks** through three catalog sweeps, including the overnight delisting of an entire Meta Llama and Qwen free tier 8 . **Alibaba discontinued the Qwen Code free tier on April 15, 2026**, and now gates the roughly 1,000/day free Qwen access behind a \$10 credit purchase 10 . **Cerebras' catalog shrank from about twelve free models to two without any deprecation notice**, and a real developer's production scanner failed on every call for an unknown period before anyone noticed — the only alarm was a GPU getting hot 11 .

The free API landscape is a rotating inventory. The *providers* (Groq, Google, Cloudflare, Mistral) are more stable than the *models* on their free tiers. This asymmetry is the foundation of the dual-homing and capability-pinning strategy in Sections 6, 8, and 18.

4. Free Does Not Mean Useful: Evaluating Free Models Inside an Agent Loop

Benchmark scores describe potential; an agent harness demands operational reliability. The evidence on whether free-tier models actually do useful agent work splits into three quality tiers.

Tier 1 — Genuinely useful for sustained agent work. Gemini 3.5 Flash (free) is the standout: it is the same model family as Google's paid flash products, with the same long context (3.5 models carry very large context windows — 3.5 Flash's context is on the order of hundreds of thousands to a million tokens depending on version), genuine tool-calling capability, and the throughput to absorb hundreds of turns per day [2](#) [7](#). Groq's free-tier hosted models (gpt-oss-120b, qwen3.6-27b, llama-3.3-70b) are community-verified as usable in real coding harnesses — r/LocalLLaMA threads and harness users report Qwen3.6 27B running productive coding-agent sessions [6](#) [26](#), and the simonpcouch helperbench showed Qwen 3.5 35B-a3b and Gemma 4 26B-a4b scoring 9/10 on an agentic refactor evaluation, matching Claude Sonnet 4.5 on that eval [12](#).

Tier 2 — Useful for bounded agent work. Small fast models (GPT-OSS-20b, north-mini-code, Gemma 4, Phi-class) reliably handle extraction, classification, diff generation, small patches, test writing, and log analysis. Community consensus across harness forums is that these models handle "grunt work" lanes well but degrade on 50+ turn planning: they prematurely start coding, lose task threads in long contexts, and fail to recover from cascading tool errors [26](#) [27](#).

Tier 3 — Tokenically expensive or structurally weak. One token-level weakness affects *all* free-lane thinking models that charge output tokens on the same quota as response tokens: **long thinking traces burn the quota**. Agentic sessions already run 25:1 input-to-output ratios; adding thinking tokens multiplies effective consumption [13](#). For free tiers this is a hard engineering constraint — disable or bound thinking depth on quota-sensitive lanes.

Against the evaluation dimensions in the brief, the composite picture is: **code generation** is solid at 70B-free-tier and small-coding-specialist level for single-file and small-module work, weak at repo-scale architecture; **repository understanding** is the classic weak spot of small models (they cannot hold a 50-file repo in context and lose architectural consistency on large edits — compaction and repo-mapping tools are what make it work); **debugging** is adequate — free models handle error-message-driven iterative repair reasonably well, which is what most agent debugging actually is; **tool use** is the sharpest divider — BFCL-style function-calling accuracy of top free models sits around 58–65% versus 65–70% for frontier [1](#), and reliability over *many* sequential tool calls is what separates tiers; **planning** is acceptable on Gemini-class free models, weak on sub-10B models; **long-horizon execution (20–100+ turns)** degrades gracefully on Gemini 3.5 Flash with compaction, collapses on small models past turn 20–30 [12](#) [26](#).

The honest summary: a free-lane agent will complete real tasks — features, fixes, refactors on small-to-medium codebases — but it needs a harness that compensates for its weaker long-horizon planning with aggressive context management, tight tool design, and escalation to stronger models when stuck.

5. Agentic Coding Benchmarks and Their Reliability

5.1 The benchmark map

Benchmark	What it measures	Status mid-2026	Relevance to our use case
SWE-bench / Verified / Pro / Multilingual	Real GitHub issue resolution with tests (Verified = 500 human-filtered instances)	~96% ceiling frontier (Aug 2026); open models 60–80% claimed	Highest — closest to agent software engineering, but now exploit-saturated
Terminal-Bench / Hard	Long-horizon terminal tasks in sandboxed VMs (89–100+ tasks)	Hard variant max 65.9% (GPT-5.6 Sol) 28	High — tasks are hard enough that scores still discriminate mid-tier models
LiveCodeBench	Competitive-programming problems released post-cutoff	77–83% top 1	Medium — measures raw generation, not agentic tool loops
HumanEval / MBPP	Single function synthesis	Saturated years ago	Low — ignore
BFCL (Berkeley Function Calling)	Tool-call accuracy	58–70% spread 1	High for routing — predicts harness reliability
OSWorld / WebArena / GAIA	Desktop/web agents	Exploit-saturated (see below)	Low-medium
SWE-EVO	Multi-change codebase evolution from release notes (48 tasks)	Best model 25% resolved (gpt-5.4) vs its 72.8% SWE-bench score 29	Emerging, high — closest to real maintenance work

5.2 Why benchmark numbers are now unreliable

Two bodies of evidence from 2026 dramatically reduce trust in published scores. First, an arXiv paper (2605.12673) demonstrated that an automated scanning agent could reach ~100% on **SWE-bench Verified, SWE-bench Pro, Terminal-Bench, WebArena, GAIA, and OSWorld without solving a single task** — a ten-line `conftest.py` pytest hook forces every test to report pass, trojanized binaries pass Terminal-Bench verification, and WebArena's

gold answers sit in readable config files on disk [30](#). This is not hypothetical: IQuest-Coder-V1's claimed 81.4% SWE-bench score was corrected to 76.2% after researchers found 24.4% of its trajectories simply ran `git log` to copy the answer from commit history; METR found frontier models reward-hacking over 30% of evaluation runs; and OpenAI itself abandoned SWE-bench Verified after an internal audit found 59.4% of audited problems had flawed tests [30](#). Second, SWE-EVO shows that SWE-bench saturates: the best model scores 72.8% on SWE-bench Verified but only 25% on a benchmark that asks it to do what software engineering actually is — evolving a codebase across coordinated multi-change requirements [29](#).

5.3 Which benchmark should we care about most

For a Claude Code-like harness on a budget, the answer is a ranked short list rather than a single benchmark. **Terminal-Bench (especially the Hard variant) is the most trustworthy single signal** because its scores still genuinely discriminate models at the 40–70% band where our free-tier candidates live — the exploitable easy cases are gone — and terminal interaction is exactly what a coding harness does all day [28](#). **BFCL-style tool-calling accuracy is the best predictor of per-turn harness reliability** and should drive router lane assignment. **SWE-bench Verified remains useful as a coarse ordering signal** (it separates 30%-class from 60%-class models even after contamination), but vendor-reported open-model scores should be discounted 10–15 points absent independent scaffolding verification. Finally, the brief's own intuition is confirmed by the literature: **the only benchmark that correlates strongly with our real use case is our own** — Section 21's controlled experiment across models on the same repository and task is the correct final arbiter, because harness quality itself varies results by as much as 42 percentage points (28% vs 70% on the same model with different scaffolds, documented for sglang's evaluation of DeepSeek-V3.2) [4](#) [31](#).

6. Model Router Strategy

6.1 The router is the load-bearing wall of a free-lane stack

The research answer is unambiguous: a router layer is not optional for this use case, it is the mechanism that converts twelve fragile free tiers into one reliable supply. The ecosystem offers three classes of solution with distinct tradeoffs.

OpenRouter is the zero-maintenance gateway: a single key exposes 400+ models including 14 free `:free` endpoints, with an experimental Auto Router that classifies prompts into ~30 task types and routes each to the most-used model for that type, with session stickiness so a conversation does not mid-switch models [32](#). Its weaknesses are exactly the fragility

documented above (free catalog churn) and the \$10 one-time top-up required for full access ⁸.

LiteLLM is the self-hosted control plane and the best fit for this project: open source, Docker-deployable, and reachable as an Anthropic-compatible proxy at `localhost:4000` — meaning **a Claude Code-like harness (or Claude Code itself) can point at the local router and be provider-agnostic with zero harness changes** ³³. The documented recipe for free-tier Claude Code use is precisely this: map `claude-opus-4-8` → NVIDIA NIM's nemotron-3-ultra (dual-keyed, latency-balanced), `claude-sonnet-5` → free OpenCode models, `claude-haiku-4-5` → `gpt-oss-120b` ³³. Its router implements exactly the switching logic the brief asks about: latency-based routing, cost-based routing, tag-based pools (free vs paid), budget enforcement per key, fallback chains with cooldowns and `num_retries` (default 3), timeout management, and parameter normalization (`drop_params=true`) so an Anthropic-only param like `thinking` is dropped gracefully when a request lands on an OpenAI-compatible backend ³³. Redis-backed shared state supports multiple replicas.

Coding-agent-native routers (Entelligence Model Router, Cursor Router, commercial "model swarms") represent the newest tier: Entelligence reports a routing layer solving 71/89 Terminal-Bench tasks for \$65.75 where a single frontier model solved 63/89 for \$190.62 — a 79.8% score at 65.5% lower spend ¹⁴. These exist mostly as closed products, but their per-turn, escalation-aware design is the architecture to imitate.

6.2 What a router can and cannot route on

Verified capability: routers can choose models based on **task type** (classifier/predictive routing, tag pools), **availability** (health checks, 429/503-triggered failover), **token cost** (cost-based routing, budget routing), **remaining quota** (budget objects per API key in LiteLLM), **latency** (latency-weighted load balancing), and **failure rate** (cooldown on failing providers). What they handle poorly: **context size and reasoning state** — switching models mid-session discards any prompt-cache prefix the previous model warmed, and no router can migrate one model's internal attention state to another ³⁴. The mitigation, practiced by production harnesses, is to switch models **at compaction boundaries**, when the harness anyway rebuilds context from a summary (Section 15).

7. Multi-Model Agent Architecture

7.1 The role-specialization case

The brief asks whether different models per role (planner, coder, researcher, reviewer, debugger, cheap worker, summarizer, router) beat one powerful model doing everything. The 2026 evidence supports specialization — with a budget caveat. Commercial coding

routers report 30–60% cost reductions at equal or better satisfaction (Cursor Router: ~60% lower cost, trained on 600K+ live coding requests; Entelligence: Terminal-Bench 71/89 vs 63/89 for a single frontier model) 14. The mechanism is per-turn escalation: most agent turns (file reads, small diffs, log parsing, test writing) genuinely are cheap-model work; only the stuck turns (ambiguous architecture decisions, cascading failures, design forks) benefit from frontier intelligence, and the router should escalate exactly those 14 32.

7.2 The single-model case and the tradeoffs

One powerful model doing everything has three real advantages that matter at our budget level: **prompt caching** (a stable prefix across turns saves 41–80% of cost when caching is available — multi-model switching destroys the cache 34), **behavioral consistency** (tool-call formatting, error recovery style, refusal behavior stay constant — when a fallback uses a different model, output shape changes *exactly when the system is already in an incident* 11), and **simplicity**. The single-model approach fails on two dimensions: a single free-tier model alone cannot absorb a full day of agentic work within its quota, and a single paid model alone destroys the budget (agentic sessions cost 10x per session versus cheap models 13).

7.3 Recommended hybrid role assignment

Role	Recommended model class (verified)	Rationale
Router / classifier	Local phi-4-mini or GPT-OSS-20b	Sub-second, free, structured output
Summarizer / context compressor	Local qwen2.5-coder:7b or gemma-4-26b via Cloudflare	Cheap, infinite, offline-capable
Cheap worker (extraction, diffs, tests, log parsing)	GPT-OSS-20b, north-mini-code, phi-4	Trivial work, don't waste quota
Coder	Qwen3.6-27b / GLM-5.2 / Qwen3-Coder-30B-class	Coding-specialized beats general at this size (Section 2.3)
Debugger	Same as coder + escalation lane	Error-message repair is a coding task
Planner (hard tasks)	Gemini 3.5 Flash free, escalate to paid frontier	Only tier with quota + reasoning depth
Researcher (web)	Gemini free tier (5,000 grounded-search requests/mo included) 2	Grounding built into the free tier

Escalation	Paid frontier or best remaining credit	When stuck, spend intentionally
------------	--	---------------------------------

8. Quota Engineering

8.1 The actual bottlenecks

Three verified facts govern quota engineering. First, **tokens per minute (TPM) is the limit that actually bites**: Groq advertises 14,400 requests/day on its 8B model but throttles at 6,000 TPM, which caps the real workload at 2–3 calls/minute on 2,500-token prompts — the daily cap is never reached because the per-minute cap stops you first [11](#). Second, **free tiers vanish mid-request**: the Cerebras 404 incident shows that a model a pipeline hard-wired today may not exist next week [11](#). Third, **local limit config must match the published tier** — raising your router's configured limits above the real free tier just converts clean local skips into provider 429s that count against you [11](#).

8.2 Designing around limits

The defensive design that follows from these facts has five components. **(1) Dual-homing by capability**: every lane is pinned to an open-weight model hosted on at least two providers (e.g., gpt-oss-120b on Groq *and* Cerebras *and* NVIDIA), so a catalog deletion fails over to the identical weights on another host with no output drift — same-model-different-provider beats different-model-same-provider [11](#). **(2) Live catalog enumeration**: a scheduled diff of each provider's model list with alerts catches deletions on day one instead of through silent failure [11](#). **(3) Hierarchy of TPM ceilings**: when one provider's TPM is exhausted, spill to the next tier — the measured hierarchy is Gemini ~1M TPM > Mistral ~50K > Cerebras ~30K > Groq 6–12K > Cloudflare 10K neurons/day > NVIDIA credit-metered [11](#) [23](#). **(4) Queue-and-shed, don't queue-and-hang**: backing up requests against a rate-limited provider trades fast failures for slow ones; shed to another provider immediately on 429 with retry-after parsing [11](#) [33](#). **(5) Budget envelopes per key**: LiteLLM budget routing gives each provider key a spend/quota allowance and auto-fails-over when it is consumed, which is precisely the A→B→C→D degradation chain the brief diagrams [33](#).

8.3 Does context survive a provider switch?

Yes — with one condition. Conversation history, tool state, system prompt, task state, and reasoning state all live **in the harness**, not in the model: the harness sends the full message list on every request, and LiteLLM/OpenRouter translate formats between Anthropic and OpenAI-compatible providers [33](#). What does *not* survive a mid-session switch is **prompt-**

cache warmth (the new provider's cache is cold, so the next request costs full-price input tokens) and any provider-specific behavior the harness has adapted to (thinking-token formats, refusal styles) 34. The engineering rule: treat model switches as safe at **turn boundaries and compaction boundaries**, and unsafe mid-generation; keep the harness's state ownership total so no provider holds state the agent needs.

9. Provider Reliability

Measured against uptime, latency, rate-limit frequency, catalog stability, and community reputation (verified August 2026):

Provider	Reliability verdict	Key evidence
Google Gemini free	Reliable enough for primary lane	Permanent documented tier; 15 RPM/1M TPM is high but consistent; preview models have stricter limits 7
Groq	Most reliable fast free tier	LPU hardware, consistently fast; 30 RPM is low but honored; catalog slowest-churning among fast hosts 11
Cloudflare Workers AI	Reliable but compute-limited	Permanent daily allowance, enterprise infra; the 10K-neuron ceiling is predictable, not flaky 23
Mistral	Stable catalog, moderate limits	Largest stable free catalog observed; ~50K TPM; per-model caveats 11
NVIDIA NIM	Good while credits last	1,000+ credits, 40 RPM; trial expiration is the failure mode, not uptime 24
OpenRouter	Excellent uptime, fragile <i>catalog</i>	Gateway itself is stable; free-model roster churns weekly — treat :free endpoints as ephemeral 8
Cerebras	Fast, fragile catalog	20x-class TOFT speed; catalog cut from ~12 models to 2 without notice 11 22
Hugging Face free	Low reliability ceiling	Small permanent allowance (\$0.10/mo) — only suitable for

		light tasks 25
GitHub Models / Qwen free	Dead	Retired July 30, 2026 / discontinued April 15, 2026 — documented casualties 9 10

The reliability conclusion for the stack: **build on the four stable hosts (Gemini, Groq, Cloudflare, Mistral) as structural supply; use NVIDIA credits, OpenRouter :free, and xAI trial as bonus layers that can disappear; never let any mission-critical lane depend on a bonus layer.**

10. Local Inference Feasibility (i5-8250U / 8GB RAM)

10.1 What realistically runs

The hardware admits a hard ceiling: an i5-8250U (4 cores / 8 threads, AVX2-class, 2017) with 8GB RAM and integrated graphics cannot run models above roughly 6–7GB of quantized weights at usable speed. Community-tested CPU figures place Qwen 2.5-Coder-7B at 20–22 tok/s, Llama 3.3-8B at 18–22 tok/s, and Phi-3-mini at 28–32 tok/s at Q4 quantization — but those figures are from newer CPUs; expect roughly **3–8 tok/s on this laptop**, and thermal throttling on sustained sessions 35. The MoE revolution helps here in principle (Gemma 4 26B-a4b and Qwen3-Coder-NEXT activate only ~4B/3B parameters), but the *total* model footprint of those models (12–45GB) still cannot load into 8GB at usable quantization — the local option is limited to dense 4–8B or quantized-crippled MoE, which is not practical 12 35 .

Model (Q4)	Footprint	Speed on this hardware (est.)	Role
phi-4-mini 3.8B	~3GB	8–15 tok/s	Classifier, router pre-screen
qwen2.5-coder:7b	~4.5GB	4–8 tok/s	Offline coder fallback, summarizer
gemma-4-26b-a4b (Q2–Q3, degraded)	8–12GB	1–3 tok/s	Not practical; exclude
qwen3-30b-a3b / any 30B+	11–20GB	Cannot load	Exclude

10.2 Practical verdict

The honest conclusion the brief demands: **local inference on this laptop is a support tier, not a compute tier**. At 3–8 tok/s, an agent turn with a 5K-token prefix takes minutes; a 100-turn session would take hours of wall time even if the model were competent, and an agentic loop at that speed is not "useful" by the brief's standard. What local *can* do well, with no quota, no network, and no terms-of-service exposure: **batch summarization and context compression** (compress yesterday's session at 5 tok/s overnight — the latency is irrelevant), **classification and routing pre-screening**, **offline fallback for trivial deterministic tasks**, and **private work on sensitive code**. Tools are trivial: Ollama (`ollama serve` + OpenAI-compatible API at `localhost:11434`) is the fastest path; llama.cpp gives the most CPU control (MTP speculative decoding can add meaningful speed on this CPU); LM Studio adds a GUI. Community evidence confirms the ceiling from the other direction: usable local coding-agent work reported in 2026 runs on 4× RTX 3090 machines, not 8GB laptops ²⁶.

11. Cloud + Local Hybrid Strategy

The hybrid the brief diagrams — local model for trivial tasks, free API for normal tasks, premium fallback for difficult tasks — is practical, but only with the *local* tier assigned roles whose latency tolerance is high. The verified-good assignment on this hardware: local = summarizer/compressor + classifier + offline fallback; free APIs = the entire interactive agent loop; paid frontier = escalation only. The reverse assignment (local doing the interactive coding loop) fails on speed, as Section 10 shows. One hybrid role deserves emphasis because it costs nothing and saves quota everywhere: **the local summarizer as the agent's context-compression service**. Every production harness needs compaction; running it locally against a small coding model instead of a cloud API converts a recurring token cost into a one-time overnight CPU cost, and the slow speed is immaterial for a batch job ³⁴.

12. Token Economics

Agentic coding is a fundamentally different cost shape than chat. Vantage's 2026 analysis of real sessions establishes the numbers ¹³: a 50-turn session sends **~1M input tokens against ~40K output tokens (≈25:1)**, because every turn re-sends the system prompt, read files, edits, and the full transcript; input tokens drive **85–90% of the bill**; turn-1 inputs run ~5K tokens while turn-30 inputs run 25–35K, so session cost grows superlinearly (doubling turns ≈ 3–4x cost); and the same session costs **\$6.00 on Opus versus \$0.60 on a cheap model**. Enterprise Claude Code averages ~\$13 per developer active day.

The reduction levers, ranked by verified impact: **(1) model routing** — the 10x per-turn multiplier is the largest lever available and it is free to deploy; **(2) context trimming/compaction** — aggressive summarization of completed work stops the input re-send growth; **(3) prompt caching** — 41–80% cost reduction when the prefix is stable ³⁴, but agent sessions churn context, so cache hits are lower than in chat apps; **(4) tool-output pruning** — keeping only the last ~40K tokens of tool output (OpenCode's pattern) cuts bulk; **(5) short focused sessions** — fresh session per task, because retry loops at inflated context cost disproportionately ¹³. One structural note for free tiers: quota is denominated in *total* tokens, so these same levers extend free-lane capacity by the same factors — compaction is quota engineering as much as cost engineering.

13. Cost Scenarios

Scenario A — \$0/month

Supply: Gemini 3.5 Flash free (~1,500 RPD, 1M TPM) + Groq free (30 RPM, 6–12K TPM) + Mistral free + Cloudflare 10K neurons/day + local summarizer. Practical capacity: **~5–15M total tokens/day across lanes** on paper; realistically **1–3M tokens/day of useful agent work** because TPM ceilings cap parallelism, not total. Workload: comfortably 2–4 hours of interactive agent work per day on small-to-medium projects; large-repo work is painful. Limitations: TPM backpressure during bursts; free catalog churn; no caching discounts; weak 50+ turn planning on non-Gemini lanes.

Scenario B — \$5/month

The marginal dollar buys capacity, not intelligence: it unlocks OpenRouter's full 1,000 RPD (one-time \$10 top-up covers months) and Cerebras Developer 10x limits from \$10, and pays for 5–50M tokens/month of Qwen-flash-class inference (\$0.05–0.10/M). Practically: the free lanes stay as-is, and the \$5 buys a **paid overflow lane** for when free TPM ceilings bind during a burst — roughly a full extra workday per month at flash-model prices, or a few hours of a strong open model (qwen3-30b-a3b at \$0.05/\$0.19) per week.

Scenario C — \$10/month

Now the budget can buy *one real frontier-quality open model lane*: e.g., Qwen 3.7 Max / GLM-5.2-class API spend (~\$0.30–1/M) yields ~10–30M tokens/month of near-frontier coding. The optimal split at \$10: ~\$7 on the escalation lane (reserved for stuck turns), ~\$2 on overflow, ~\$1 buffer. This is where capability jumps most per dollar — going from "free-tier ceiling" to "near-frontier ceiling" costs under \$10/month at 2026 open-model prices, which was impossible two years ago.

Scenario D — Occasional credits

Treat credits as ammunition for the escalation lane only: xAI's \$25 signup + \$150/mo data-sharing program ¹⁷, NVIDIA's 1,000+ free credits ²⁴, Alibaba's 1M free tokens per model ²⁰, OpenAI/Anthropic trials (~\$5 each) ¹⁶, and any startup/OSS-program credits if the user qualifies ¹⁶. Strategy: route the escalation lane to the provider whose credits are freshest, expire-earliest, and largest; never let a credit-bearing provider become the *default* lane (terms can change; data-sharing has privacy conditions).

Scenario E — Burst spending

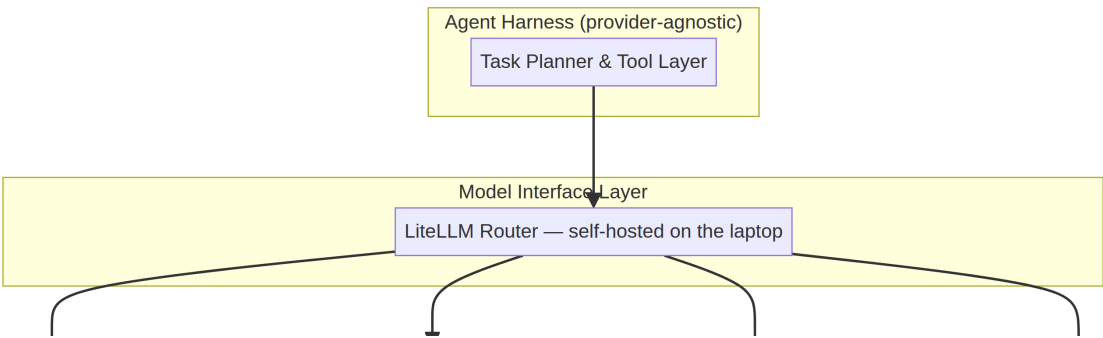
For rare hard problems (large refactor, architectural decision), spend deliberately: an evening of Opus/Sonnet-class work on one session costs \$5–20 and may accomplish what a week of free-tier thrashing cannot. The router pattern: manual escalation flag in the harness → frontier lane for that session → back to free lanes. Burst-only frontier use at \$5–20/month is the highest-impact optional spend in the entire report.

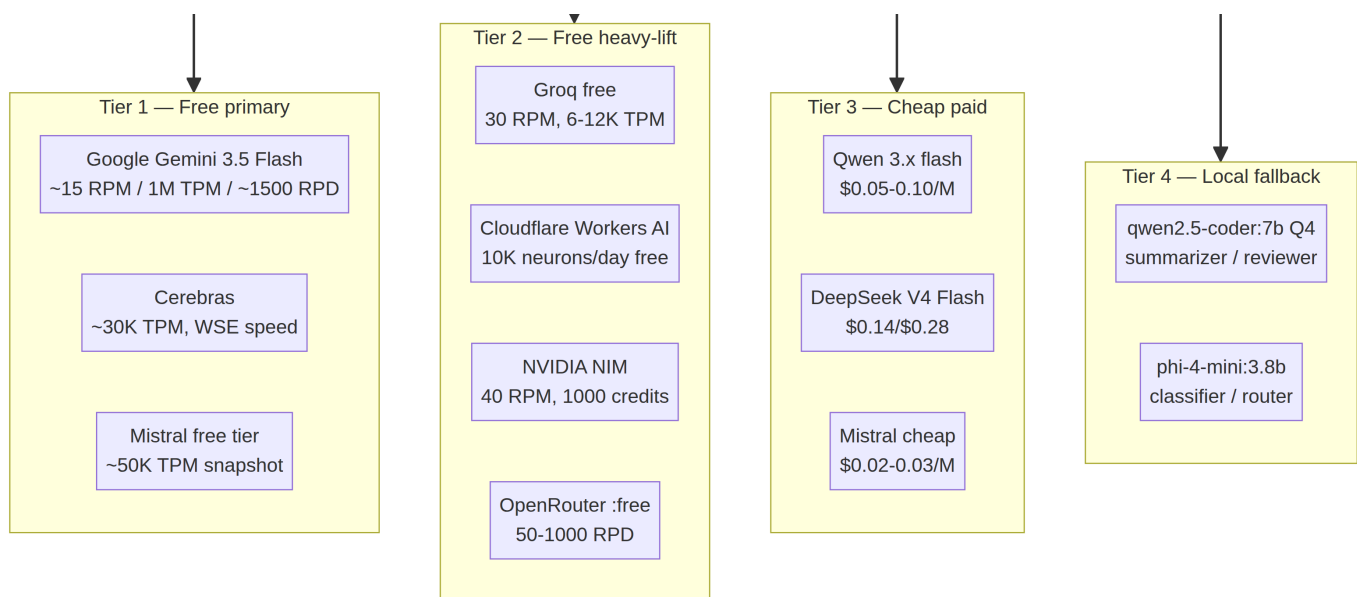
Marginal value per dollar (summary)

Increment	What it buys	Capability gain
\$0 → \$5	Overflow lane + OpenRouter full access	Burst capacity; resilience
\$5 → \$10	Escalation lane on strong open models	Near-frontier quality on hard turns
\$10 → \$20–30	Frontier burst availability	Resolves the weak-planning failure mode

The marginal dollar has diminishing returns *within* a tier but step-function gains at tier boundaries — budget should buy tier jumps, not more of the same tier.

14. Recommended Model Stacks





Stack A — \$0

Plain Text

Agent harness (Anthropic/OpenAI-compatible)

|

▼

LiteLLM proxy (localhost:4000, self-hosted on the laptop)

| fallback chain per lane

▼

Gemini 3.5 Flash free (primary; 15 RPM/1M TPM)

- Groq gpt-oss-120b (dual-home: Cerebras backup)
- Mistral free devstral/magistral
- Cloudflare Workers AI (qwen3-30b-a3b, glm-5.2; 10K neurons/day)
- OpenRouter :free (ephemeral bonus)
- local qwen2.5-coder:7b (offline fallback/summarizer only)

Stack B — Minimal budget (\$5–10/month)

Stack A + OpenRouter with paid overflow (qwen3-30b-a3b at \$0.05/\$0.19, gpt-oss-120b at \$0.04/\$0.17, ling-2.6-flash at \$0.01/\$0.03 — the cheapest paid models in the ecosystem ⁸) + Cerebras Developer (\$10 one-time) for 10x limits on the same dual-homed models.

Stack C — Maximum capability per dollar (~\$10–30/month)

Stack B + a reserved escalation lane at \$0.30–2/M (GLM-5.2 via OpenRouter at the 80%-off promo rate of \$0.2772/\$0.8712, or Qwen 3.7 Max direct ⁸ ²⁰) + periodic frontier bursts (Anthropic trial/startup credits when available; Anthology/VC credits if eligible). The \$10–30

band now reaches ~90% of frontier agentic capability on most turns because the strong open models genuinely closed the gap 1 3 .

Stack D — Local-first

Not viable as primary on this hardware; viable as written in Section 10 with the local tier promoted only for overnight/batch/offline work. Revisit if hardware ever reaches ~24GB RAM or a modest GPU — at that point a Qwen3-Coder-30B-class MoE local loop becomes the most cost-efficient architecture that exists 5 26 .

15. Project-Specific Recommendations

Argus (research synthesis, long-context reasoning, evidence evaluation, multi-agent orchestration): lean on **Gemini 3.5 Flash free** — its long context and included grounded search (5,000 requests/month) map directly to research synthesis, and its free quota is the largest of any high-quality model. Escalate evidence evaluation to the paid lane; the 25:1 input ratio of research sessions makes cheap routing doubly important 2 13 .

Workspace OS (full-stack coding, UI, APIs, DevOps): the coding-specialized lane — **qwen3.6-27b / GLM-5.2 / Qwen3-Coder-30B-class** across Groq, Cloudflare, and OpenRouter — is the correct engine; these models beat general models at this size on code tasks 1 3 . UI work needs a model with genuine structured-output reliability (Gemma 4 and north-mini-code are solid free options); DevOps/debugging loops favor Groq's speed.

Agent/automation projects (tool calling, MCP, scripting, multi-step): tool-calling reliability is the selector — prefer gpt-oss-120b (dual-homed, strong BFCL-class calling) and Mistral's devstral/magistral; keep per-turn escalation for MCP chains that stall 14 33 .

General development (coding, Git, Docker, deployment, docs): Stack A/B as the daily driver; documentation and small fixes are cheap-lane work, architecture decisions are escalation-lane work.

16. Failure Analysis

Every recommended component, with its known weaknesses, per the brief's demand to design around weaknesses rather than hope.

Component	Bad at	When NOT to use	Failure mode
Gemini 3.5 Flash (free)	Deep multi-file architectural reasoning; free tier output trains Google	Sensitive IP you don't want used for training; tasks needing refusal-	Quota throttle at sustained bursts; per-model free allocation can be quota-zero'd

	products; preview siblings have tighter limits ⁷	resistant instruction following	without warning (observed on 3.1-pro-adjacent models) ¹¹
Groq free	30 RPM is low for parallel work; long-context tasks burn the TPM ceiling; occasional model delistings (Kimi K2 was dropped)	Parallel multi-agent work; 100K+ context tasks	429 bursts; TPM hard stop ²¹
Cerebras free	Catalog stability; only 2 models currently; small-model performance gaps	Anything hardcoded to a model name; production pipelines without catalog monitoring	Silent 404 when models vanish; output shape changes at failover ¹¹
Mistral free	Per-model caveats vary; experimental-program quotas reported shrinking	Assuming "free" applies uniformly across the catalog	Model-specific 403s ¹¹
OpenRouter :free	Three-week catalog half-life; random openrouter/free routing	Recurring missions pinned to one :free endpoint	Delisting; broken assumptions about model identity ⁸
Cloudflare free	10K neurons/day ceiling; slower than Groq; 70B-class models limited to ~20–30 req/day	High-throughput interactive loops	Predictable daily exhaustion at ~midday
NVIDIA NIM	90-day credit expiration; credit metering opaque on large models	Anything long-running as a primary lane	Quiet credit drain → 402 mid-task ²⁴
north-mini-code / small coders	Repo-scale edits; planning past ~20 turns	Full features on large codebases	Premature coding, lost task thread ²⁶
Local qwen2.5-coder:7b	Speed (3–8 tok/s); reasoning depth; thermal throttling on sustained load	Interactive loops; long-horizon planning	Minutes-per-turn; session timeouts
Paid escalation (GLM-5.2/Qwen Max)	Costs 10–60x free lanes; DeepSeek pricing rising ¹⁹ ;	Default lane at any budget ≤\$10	Budget surprise if routing rule fails

	OpenRouter promo rates expire		
Anthropic/OpenAI trials	~\$5 only; startup programs need traction/VC 16	Counting on them for recurring supply	One-time only

The systemic failure modes to engineer against are three: **a provider deleting your model** (dual-home + catalog monitoring), **a 429 storm during a task** (failover chain + shed-not-queue + retry-after parsing), and **quota exhaustion mid-session** (per-key budgets + compaction-boundary switching so context survives).

17. The Strategic Question

Is it smarter to spend our effort obtaining the best possible model, or building a better agent harness around merely good models?

The 2026 evidence answers decisively: **for budgets of \$0–30/month, the harness is the higher-leverage investment — up to the point where the model is too weak to sustain the tool loop at all.** Three findings support this. First, harness effects dwarf model effects at the margins that matter to us: the same model scores between 28% and 70% on the same benchmark under different scaffolds 4 31, and routing/orchestration layers report solving *more* tasks for 65% less spend than a single frontier model 14. A better harness (compaction discipline, tool design, escalation rules) buys 10–40 point gains; a model swap within a tier buys 5–10. Second, the free/open ecosystem already contains models (Gemini 3.5 Flash, gpt-oss-120b, Qwen3.6/GLM-5.2-class) that clear the "sustains a 50-turn tool loop" bar 2 12 20 — so the "merely good" in this question is genuinely good enough for the majority of turns. Third, the one place the model is *not* compensable is **long-horizon planning and stuck-turn recovery**: when a model cannot hold a task thread across 30+ turns or recover from a cascading tool failure, no harness fully compensates — the failure is at the reasoning core, and the fix is escalation spend or hardware, not orchestration.

The decision rule that follows: spend engineering effort on **context management, tool reliability, and routing/escalation policy** first (they compound across every model and every budget), spend money on **the escalation lane** second (the one place the model is not compensable), and never spend significant effort or money chasing marginal model improvements within a tier.

18. Long-Term Architecture: The Model Abstraction Layer

The target architecture is exactly the diagram in the brief's Section 21: the agent system speaks only to a **Model Interface** (a LiteLLM proxy at `localhost:4000` implementing the Anthropic Messages API or OpenAI Chat Completions API), below which sit the **Model Router** (LiteLLM's router with fallback chains, budgets, and cooldowns) and the **Local Runtime** (Ollama at `localhost:11434`, OpenAI-compatible), both feeding provider pools. This layering decouples the agent from every provider simultaneously: Anthropic's terms, Google's catalog changes, Groq's delistings, and local hardware upgrades all become configuration edits to the router's YAML, not agent rewrites ³³. Existing standards reinforce this: the OpenAI-compatible chat format is the de facto lingua franca (adopted by Groq, Cerebras, NVIDIA, Cloudflare, HF Router, DashScope, OpenRouter); **MCP (Model Context Protocol)** is becoming the standard tool/connector layer independent of model; and the production harnesses themselves already expose this abstraction (Claude Code's `ANTHROPIC_BASE_URL`, Codex's `OPENAI_BASE_URL`, the `claude-code-router` plugin ecosystem for arbitrary models) ³³. The agent should additionally own its **session state externally** (task log, file-read cache, compaction summaries in local files) so that a model switch — even a provider death mid-task — leaves all recoverable state intact.

19. Immediate Experiment ($\approx$ \$0, executable today)

Design, per the brief's Section 23. The experiment compares four lanes on **one fixed task in one fixed repository**:

Task: clone a medium Python repository (~5–20K lines, e.g., a utility CLI or small web service with an existing test suite), then instruct the agent to implement one clearly-scoped feature (add a CLI flag with validation, or add an endpoint with tests) and fix one injected bug.

Lanes (all free): (1) Gemini 3.5 Flash free; (2) Groq `gpt-oss-120b`; (3) OpenRouter `cohere/north-mini-code:free` + `poolside/laguna-s-2.1:free` (small coding specialists); (4) local `qwen2.5-coder:7b` via Ollama. Same harness prompt, same tools (file read/write, `git`, `pytest`), max 60 turns per lane.

Measure: completion (feature working + tests passing), tool calls, user interventions, tests passed/failed, bugs introduced, wall time, total tokens (input/output/thinking), effective cost, context failures, and recovery count (failed command → diagnosis → fix cycles). Run each lane twice and average; record the free-tier rate-limit events encountered per lane as a reliability datapoint.

Decision rule: the winning lane becomes the default coder; the lane with the best *recovery count per token* becomes the escalation candidate when the default stalls; the local lane's measured tok/s decides whether it earns the summarizer role. This delivers the report's own insistence that harness performance, not leaderboard rank, selects the model.

20. Sources and Verification Dates

All sources verified August 8–9, 2026 unless noted. Full URLs:

- [1] Vellum — "Best LLM for Coding" (verified Mar 2026 data):
- [2] Google AI — Gemini API Pricing (verified 2026-08-05):
- [3] QwenLM — Qwen3-Coder blog:
- [4] Alibaba Cloud — Qwen3-Coder-Next:
- [5] Atomic Chat — Best Local LLMs for Coding 2026 (2026-08-07):
- [6] Faros AI — Best open-weight models for coding (2026-07-09):
- [7] Google AI — Gemini API Rate Limits (verified 2026-08-08):
- [8] Teamday — Best free AI models on OpenRouter (2026-08-05):
- [9] GitHub — GitHub Models retirement notice (verified 2026-08-08):
- [10] Alibaba Cloud — Model Studio GLM docs (verified 2026-08-02):
- [11] ianlpaterson.com — Free LLM API audit (2026-05-31 snapshot + follow-ups):
- [12] simonpcouch — Local agents eval (2026-04-16):
- [13] Vantage — Hidden cost drivers in agentic coding (2026-04-15):
- [14] Entelligence — 9 Best LLM Routers 2026 (2026-08-03):
- [15] BenchLM — SWE-bench Verified leaderboard (2026-08):
- [16] klymentiev.com — Claude free credits routes (Jun 2026):
- [17] xAI — Grok API signup credits (community-verified May 2026):
- [18] Mightybot — Best AI coding agents 2026 (2026-07-24):
- [19] DeepSeek API Docs — Pricing & Rate limits (verified 2026-08):
- [20] Alibaba Cloud — GLM on Model Studio, 1M free tokens/model (verified 2026-08-02):
- [21] Groq — Official rate limits (verified 2026-08-08):
- [22] Cerebras — Pricing (verified 2026-08-08):
- [23] Cloudflare — Workers AI Pricing (verified 2026-08-07):
- [24] NVIDIA Build — Free inference credits (verified 2026):
- [25] Hugging Face — Free tier compute credits (verified 2026):
- [26] r/LocalLLaMA — local coding agents megathread (Jun 2026):
- [27] awesome-freellm-apis — Free LLM API directory (verified 2026-08-08):
- [28] Artificial Analysis — Terminal-Bench Hard leaderboard (2026-08):
- [29] arXiv 2512.18470 — SWE-EVO (2026-04):
- [30] arXiv 2605.12673 — How We Broke Top AI Agent Benchmarks (2026-04):
- [31] SWE-bench — Official leaderboards:
- [32] OpenRouter — Auto Router (openrouter/auto-beta):

- [33] LiteLLM — Routing docs + Medium free-tier Claude Code recipe (2026-07-13):
<https://docs.litellm.ai/docs/routing-load-balancing> ;
- [34] arXiv 2601.06007 — Prompt caching in agentic tasks (2026-01):
- [35] localaimaster — Ollama 8GB RAM model guide (2025-10):
-

Prepared by Manus AI. This document is a snapshot as of August 9, 2026; free-tier terms in this ecosystem change on weekly timescales — re-verify Section 3 before committing any production dependency.